UNIDAD ACADÉMICA MULTIDISCIPLINARIA MANTE

DISEÑO ELECTRÓNICO BASADO EN SISTEMAS EMBEBIDOS

Prácticas Bloque I

Alumno(a):

García Segura América Yanely

8° “E”

Maestro(a):

López Piña Daniel

Ciudad Mante, Tamaulipas Abril, 2026

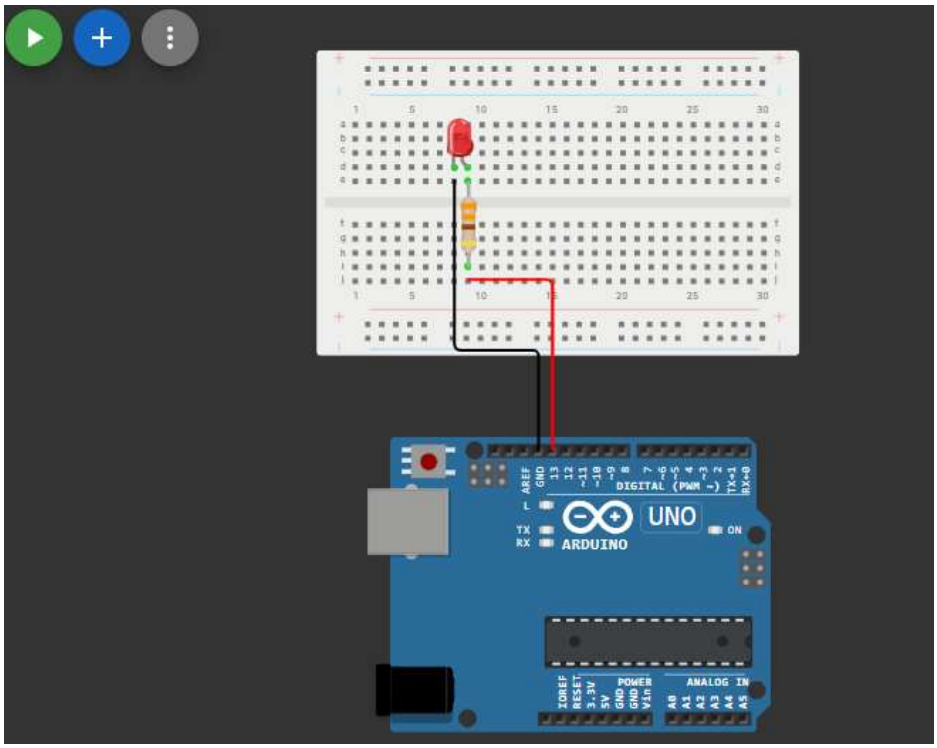
Contenido

PRACTICA 1.- Hola Mundo Embebido — Blink y Monitor Serial 4
PARTE 1: Conexión del circuito 4
PARTE 2: Sketch Blink básico 5
PARTE 3: 6
PARTE 4: Ejercicios de modificación 7
Ejercicio 1 7
Ejercicio 2 9
Ejercicio 3 12
PRACTICA 2.- Entradas Digitales — Pulsadores y Pull-up/Pull-down 14
PARTE 1: Lectura básica con Pull-down externo 14
PARTE 2: Pull-up interno y debounce por software 16
PARTE 3: Ejercicios 19
Ejercicio 1 19
Ejercicio 2 22
Ejercicio 3 24
PRACTICA 3.- Salidas Analógicas — PWM, Brillo de LED y Buzzer 25
PARTE 1: Control de brillo con analogWrite 26
PARTE 2: Efecto 'Breathing' con función seno 28
PARTE 3: Buzzer: tonos con tone() 30
PRACTICA 4.- Sensores Analógicos — ADC, LDR y Potenciómetro 32
PARTE 1: Potenciómetro: divisor de voltaje variable 32
PARTE 2: LDR: sensor de luz 34
PRACTICA 5.- Sensor de Temperatura y Humedad DHT22 37
PARTE 1: Instalación de librería y conexión 37
PRACTICA 6.- Actuadores — Motor DC y Servomotor 40
PARTE 1: Por qué usar un driver de motor 41
PARTE 2: Servomotor con librería Servo.h 45
PRACTICA 7.- Comunicación I2C — Sensor BMP280 y Pantalla OLED 48

PARTE 1: Escaneo del bus I2C 48
PARTE 2: Lectura BMP280 + Display OLED 51
PRACTICA 8.- Comunicación SPI y Proyecto Integrador Bloque I 56

PRACTICA 1.- Hola Mundo Embebido — Blink y Monitor Serial

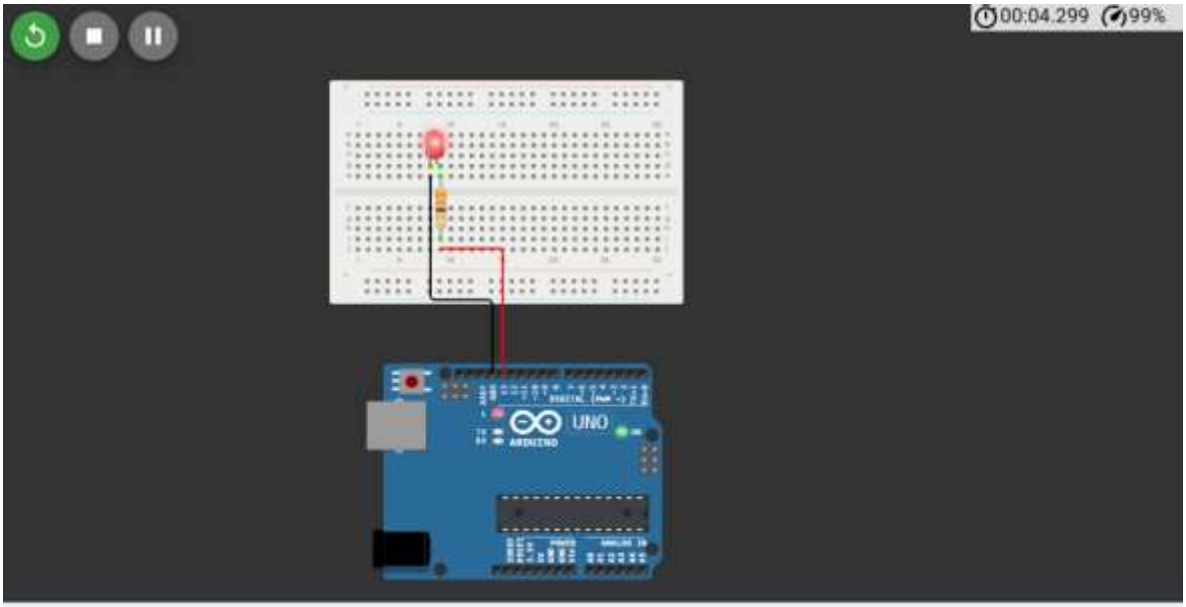
PARTE 1: Conexión del circuito



PARTE 2: Sketch Blink básico

```
const int PIN_LED = 13;

void setup() {
  // put your setup code here, to run once:
  pinMode(PIN_LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("=== LAB01 ===");
}
void loop() {
  // put your main code here, to run repeatedly:
  digitalWrite(PIN_LED, HIGH);
  Serial.println(“LED Encendido”)
  delay(1000);

  digitalWrite(PIN_LED, LOW);
  Serial.println(“LED Apagado”)
  delay(1000);
}
```

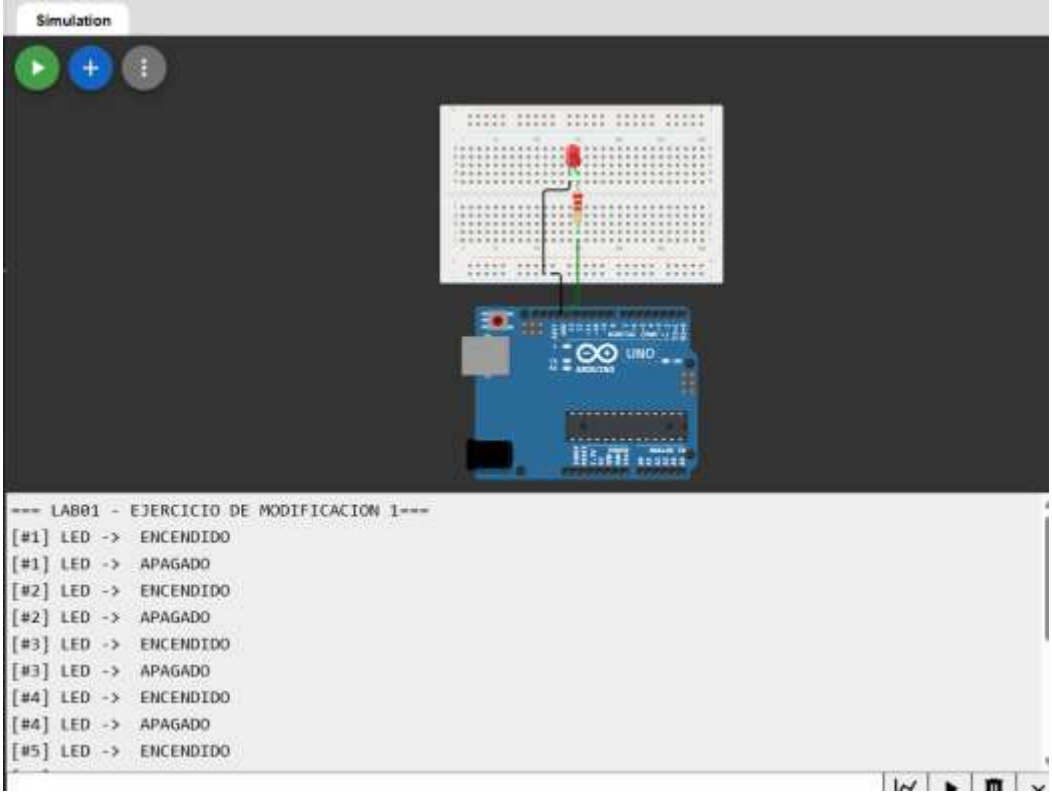
PARTE 3: Monitor Serial




```
=== LAB01 ===
LED Encendido
LED Apagado
LED Encendido
LED Apagado
LED Encendido
```

PARTE 4: Ejercicios de modificación

Ejercicio 1




```
const int PIN_LED = 13;
int contador = 0;

void setup() {
  // put your setup code here, to run once:
  pinMode(PIN_LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("=== LAB01 - EJERCICIO DE MODIFICACION 1===");
}

void loop() {
```

```
  // put your main code here, to run repeatedly:
  contador++;
  digitalWrite(PIN_LED, HIGH);
  Serial.print("[#");
  Serial.print(contador);
  Serial.println("] LED ->  ENCENDIDO");
  delay(200);

  digitalWrite(PIN_LED, LOW);
  Serial.print("[#");
  Serial.print(contador);
  Serial.println("] LED ->  APAGADO");
  delay(800);
}
```

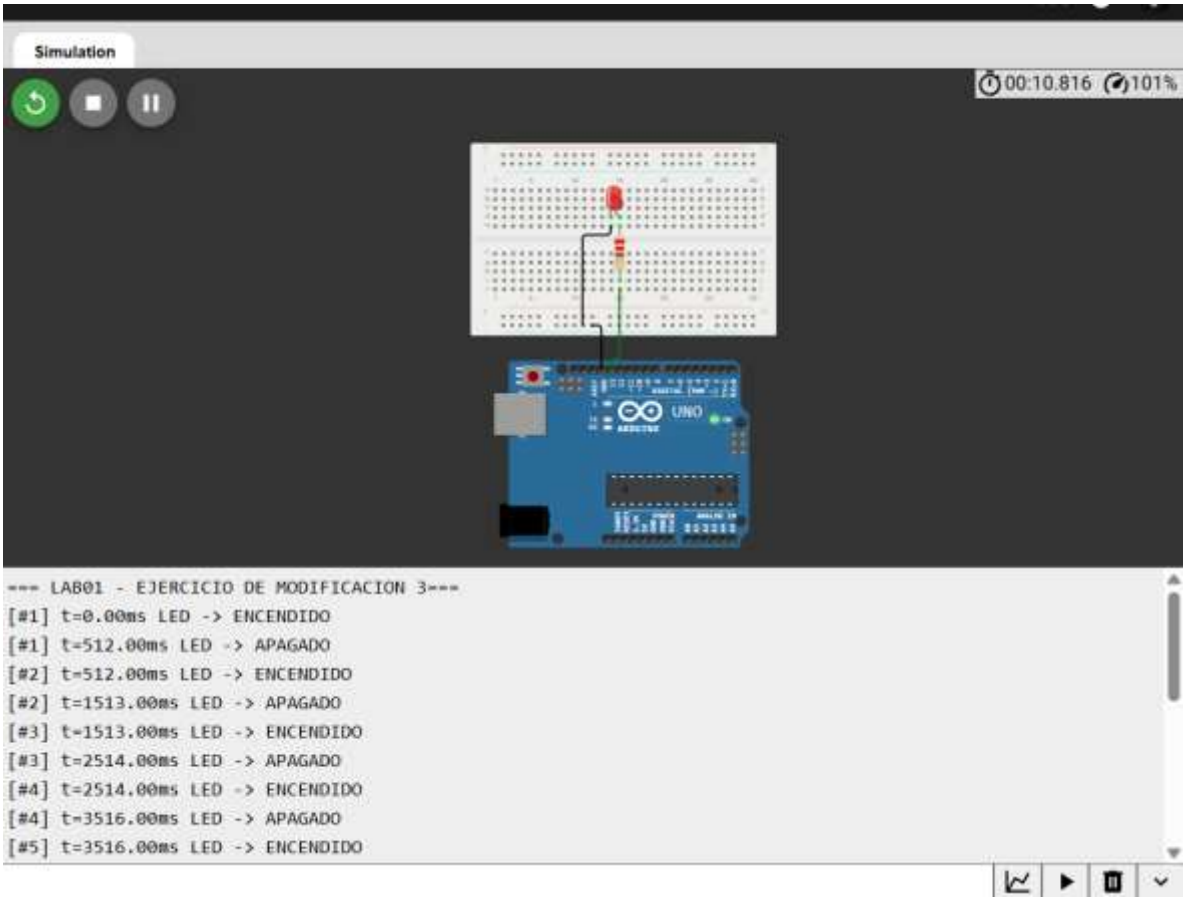
Ejercicio 2

```
const int PIN_LED = 13;
int contador = 0;

void setup() {
  // put your setup code here, to run once:
  pinMode(PIN_LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("=== LAB01 - EJERCICIO DE MODIFICACION 2===");
  Serial.println("Sistema SOS Iniciado");
  }

void loop() {
```

```
// put your main code here, to run repeatedly:
Serial.println("ENVIANDO: S");
for(int i = 0; i < 3; i++){
  digitalWrite(PIN_LED, HIGH);
  delay(200);
  digitalWrite(PIN_LED, LOW);
  delay(200);
}
delay(400);

Serial.println("ENVIANDO: O");
for(int i = 0; i < 3; i++){
  digitalWrite(PIN_LED, HIGH);
  delay(600);
  digitalWrite(PIN_LED, LOW);
  delay(200);
}
delay(400);

Serial.println("ENVIANDO: S");
for(int i = 0; i < 3; i++){
  digitalWrite(PIN_LED, HIGH);
  delay(200);
  digitalWrite(PIN_LED, LOW);
  delay(200);
}
```

```
  Serial.println("PAUSA DE CICLO...");
  delay(2000);
}
```

Ejercicio 3

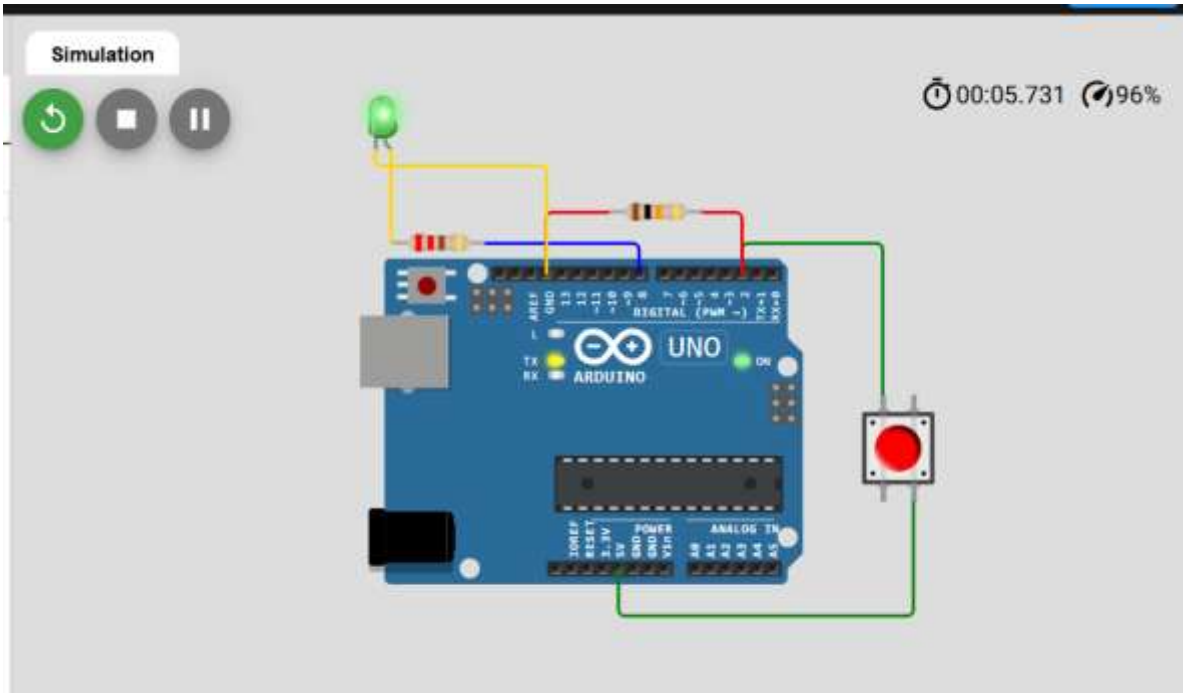



```
const int PIN_LED = 13;
int contador = 0;
float tiempoActual=0;

void setup() {
  // put your setup code here, to run once:
  pinMode(PIN_LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("=== LAB01 - EJERCICIO DE MODIFICACION 3===");
  }
```

```
void loop() {
  // put your main code here, to run repeatedly:
  contador++;

  digitalWrite(PIN_LED, HIGH);
  Serial.print("[#");
  Serial.print(contador);
  Serial.print("] t=");
  Serial.print(tiempoActual);
  Serial.println("ms LED -> ENCENDIDO");
  delay(500);

  tiempoActual = millis();

  digitalWrite(PIN_LED, LOW);
  Serial.print("[#");
  Serial.print(contador);
  Serial.print("] t=");
  Serial.print(tiempoActual);
  Serial.println("ms LED -> APAGADO");
  delay(500);
}
```

PRACTICA 2.- Entradas Digitales — Pulsadores y Pull-up/Pull-down

PARTE 1: Lectura básica con Pull-down externo

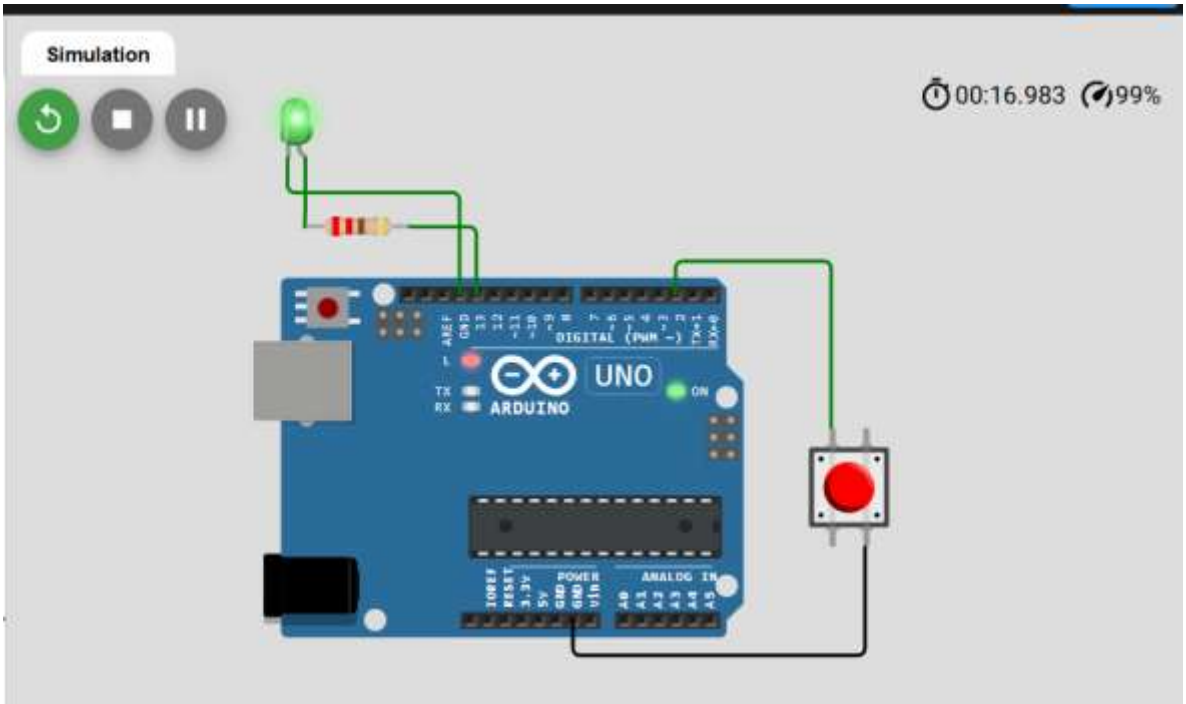



```
Boton: PRESIONADO
Boton: PRESIONADO
Boton: PRESIONADO
Boton: PRESIONADO
Boton: PRESIONADO
Boton: PRESIONADO
Boton: PRESIONADO
```

```
const int PIN_BOTON = 2;
const int PIN_LED = 8;

void setup() {
  pinMode(PIN_BOTON, INPUT);
  pinMode(PIN_LED, OUTPUT);
```

```
  Serial.begin(9600);
  Serial.println("Lab 02 — Lectura de pulsador");
}

void loop() {
  int estado = digitalRead(PIN_BOTON);

  if (estado == HIGH) {
    digitalWrite(PIN_LED, HIGH);
    Serial.println("Boton: PRESIONADO");
  } else {
    digitalWrite(PIN_LED, LOW);
  }

  delay(50);
}
```

PARTE 2: Pull-up interno y debounce por software



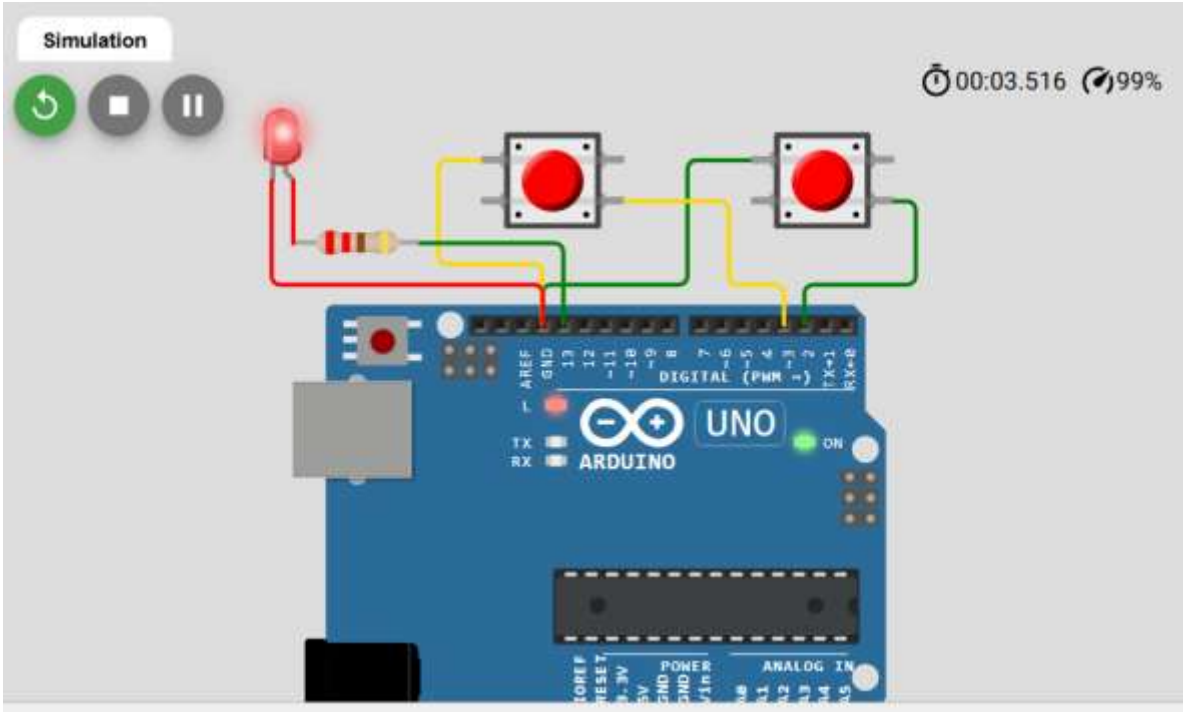
```
const int PIN_BOTON = 2;
const int PIN_LED = 13;

int estado_led = LOW;
int conteo = 0;

unsigned long ultimoTiempo = 0;
const int debounceDelay = 50;
```

```
int ultimoEstadoBoton = HIGH;

void setup() {
  pinMode(PIN_BOTON, INPUT_PULLUP);
  pinMode(PIN_LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("Lab 02 - Debounce funcionando");
}

void loop() {
  int lectura = digitalRead(PIN_BOTON);

  if (lectura != ultimoEstadoBoton) {
    if ((millis() - ultimoTiempo) > debounceDelay) {

      // Detecta cuando presionas
      if (lectura == LOW) {
        conteo++;

        estado_led = !estado_led;
        digitalWrite(PIN_LED, estado_led);

        Serial.print("Pulsacion #");
        Serial.print(conteo);
```

```
      Serial.print(" LED: ");
      Serial.println(estado_led ? "ON" : "OFF");
    }

    ultimoTiempo = millis();
  }
 }

 ultimoEstadoBoton = lectura;
}
```

PARTE 3: Ejercicios

Ejercicio 1



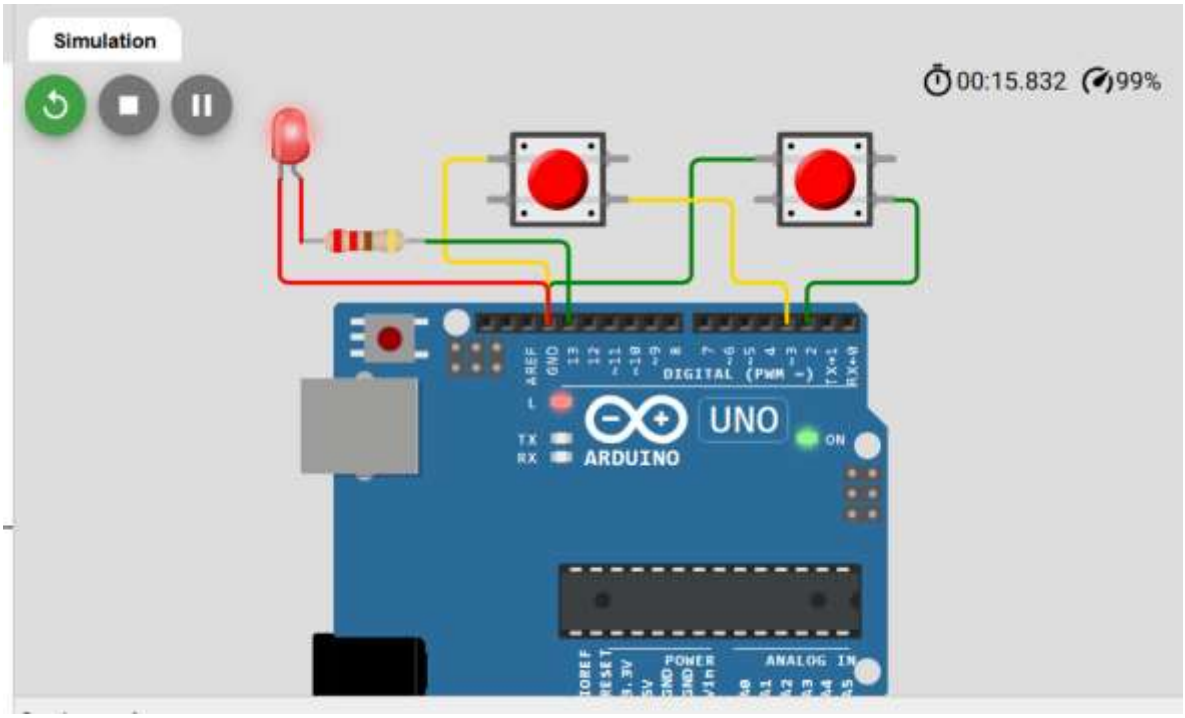

```
Ejercicio 1
Conteo: 1
Conteo: 2
Conteo: 3
Conteo: 4
Conteo: 5
Conteo: 6
CONTEO REINICIADO
```

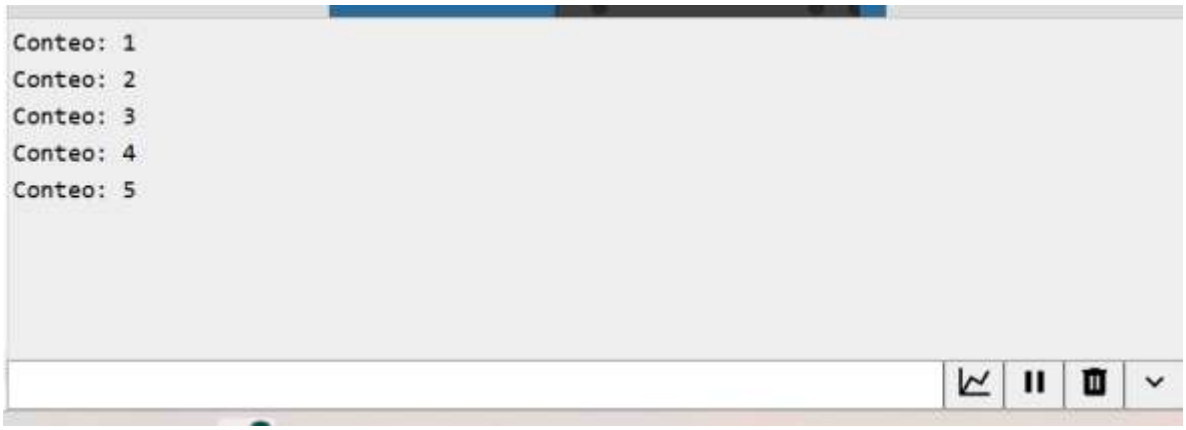
```
const int BTN_A = 2;
const int BTN_B = 3;
const int LED = 13;

int conteo = 0;
int estadoLED = LOW;
```

```
int ultimoA = HIGH;
int ultimoB = HIGH;

void setup() {
  pinMode(BTN_A, INPUT_PULLUP);
  pinMode(BTN_B, INPUT_PULLUP);
  pinMode(LED, OUTPUT);

  Serial.begin(9600);
  Serial.println("Ejercicio 1");
}

void loop() {
  int lecturaA = digitalRead(BTN_A);
  int lecturaB = digitalRead(BTN_B);

  if (lecturaA == LOW && ultimoA == HIGH) {
    conteo++;
    estadoLED = !estadoLED;
    digitalWrite(LED, estadoLED);

    Serial.print("Conteo: ");
    Serial.println(conteo);
  }
```

```
  if (lecturaB == LOW && ultimoB == HIGH) {
    conteo = 0;
    Serial.println("CONTEO REINICIADO");
  }

  ultimoA = lecturaA;
  ultimoB = lecturaB;

  delay(200);
}
```

Ejercicio 2

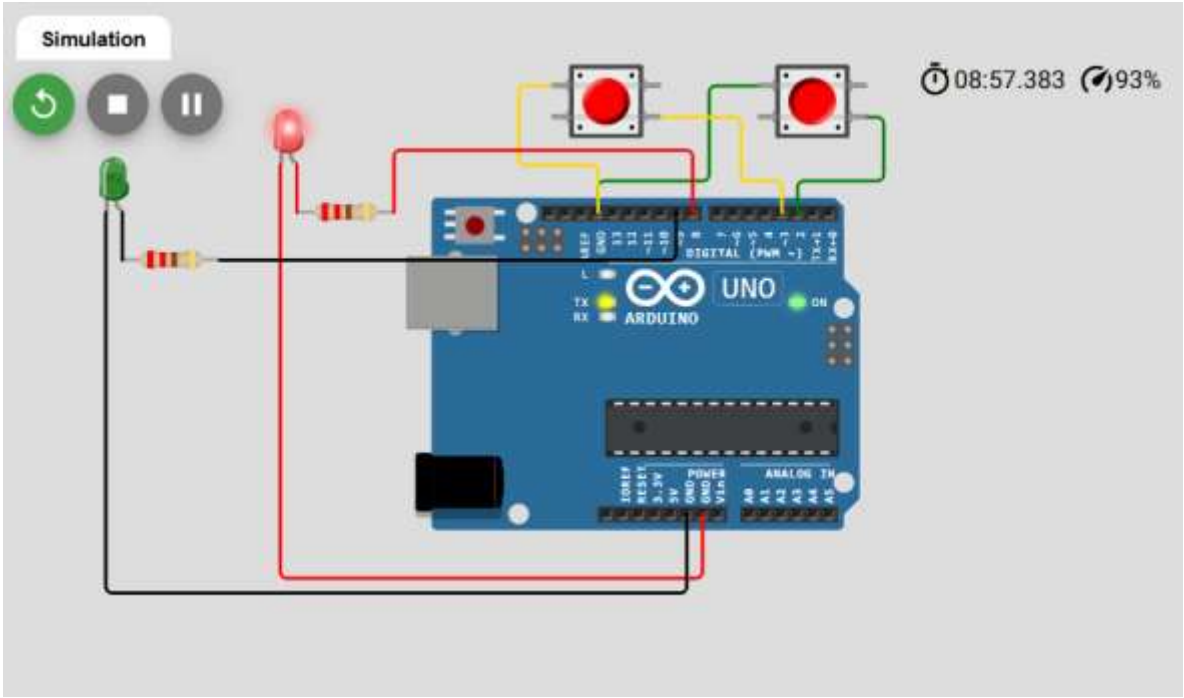





```
const int BTN = 2;
const int LED = 13;

int conteo = 0;
int ultimoEstado = HIGH;

void setup() {
```

```
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  int lectura = digitalRead(BTN);

  if (lectura == LOW && ultimoEstado == HIGH) {
    conteo++;
    Serial.print("Conteo: ");
    Serial.println(conteo);

    for (int i = 0; i < conteo; i++) {
      digitalWrite(LED, HIGH);
      delay(300);
      digitalWrite(LED, LOW);
      delay(300);
    }
  }

  ultimoEstado = lectura;
  delay(200);
}
```

Ejercicio 3



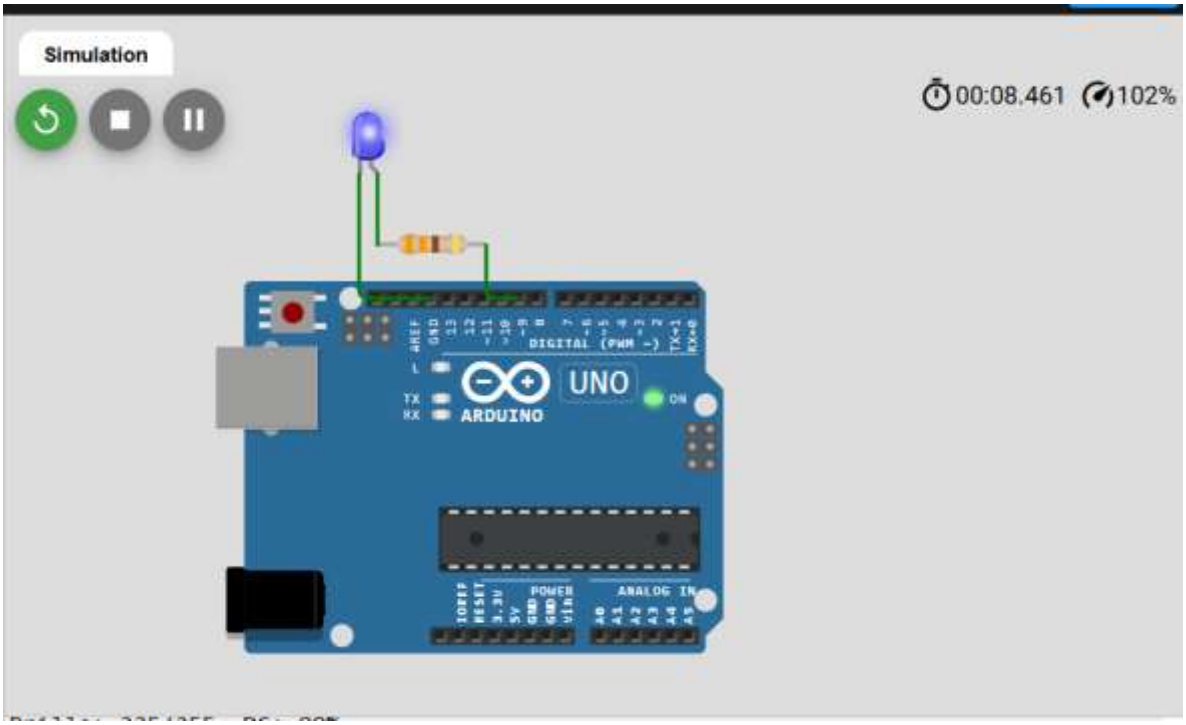

```
Boton A presionado - LED VERDE
Boton A presionado - LED VERDE
Boton A presionado - LED VERDE
Boton B presionado - LED ROJO
Boton B presionado - LED ROJO
Boton B presionado - LED ROJO
Boton A presionado - LED VERDE
Boton B presionado - LED ROJO
```

```
const int BTN_A = 2;
const int BTN_B = 3;

const int LED_VERDE = 8;
const int LED_ROJO = 9;

void setup() {
  pinMode(BTN_A, INPUT_PULLUP);
  pinMode(BTN_B, INPUT_PULLUP);
```

```
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_ROJO, OUTPUT);

  Serial.begin(9600);
}

void loop() {

  if (digitalRead(BTN_A) == LOW) {
    digitalWrite(LED_VERDE, HIGH);
    Serial.println("Boton A presionado - LED VERDE");
  } else {
    digitalWrite(LED_VERDE, LOW);
  }

  if (digitalRead(BTN_B) == LOW) {
    digitalWrite(LED_ROJO, HIGH);
    Serial.println("Boton B presionado - LED ROJO");
  } else {
    digitalWrite(LED_ROJO, LOW);
  }

  delay(200);
```

PRACTICA 3.- Salidas Analógicas — PWM, Brillo de LED y Buzzer

PARTE 1: Control de brillo con analogWrite

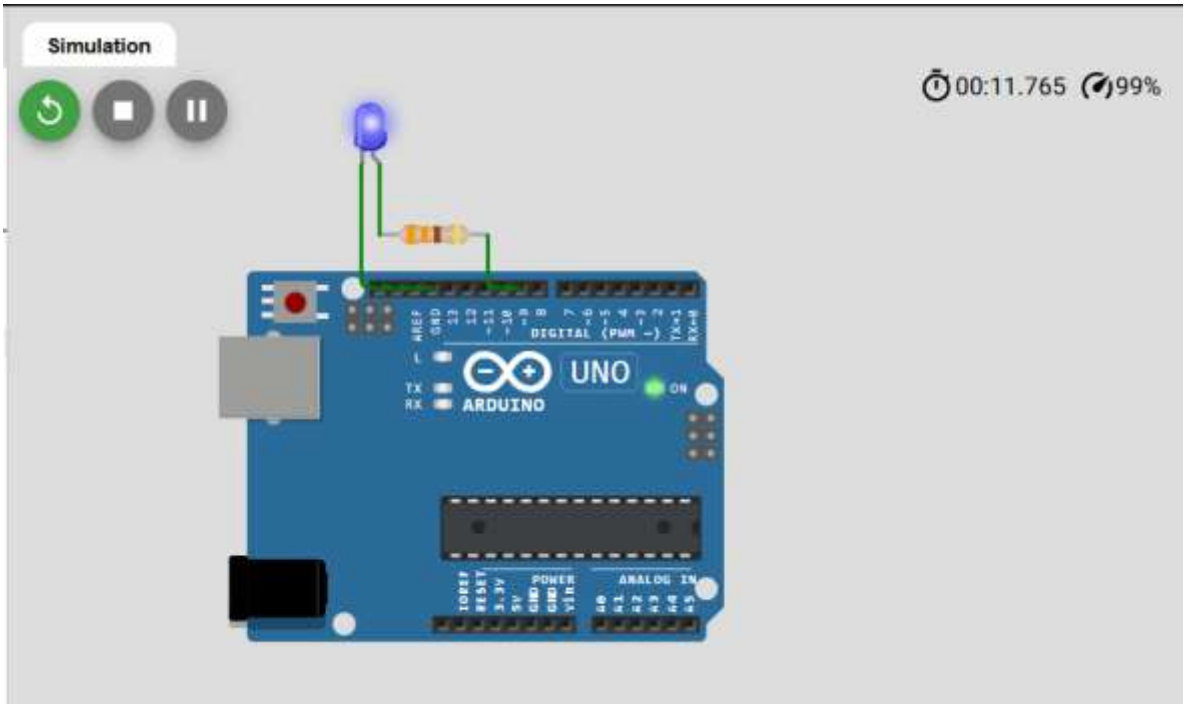



```
Brillo: 225/255  DC: 88%
Brillo: 230/255  DC: 90%
Brillo: 235/255  DC: 92%
Brillo: 240/255  DC: 94%
Brillo: 245/255  DC: 96%
Brillo: 250/255  DC: 98%
Brillo: 255/255  DC: 100%
```

```
// Lab03 PWM Brillo.ino
const int PIN_LED = 9;

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Serial.begin(9600);
  Serial.println("Lab 03 - Control PWM");
  Serial.println("Escribe un valor entre 0 y 255:");
```

```
}

void loop() {
  // Subir de 0 a 255
  for (int brillo = 0; brillo <= 255; brillo += 5) {
    analogWrite(PIN_LED, brillo);
    Serial.print("Brillo: ");
    Serial.print(brillo);
    Serial.print("/255  DC: ");
    Serial.print((brillo * 100) / 255);
    Serial.println("%");
    delay(30);
  }

  for (int brillo = 255; brillo >= 0; brillo -= 5) {
    analogWrite(PIN_LED, brillo);
    delay(30);
  }
}
```

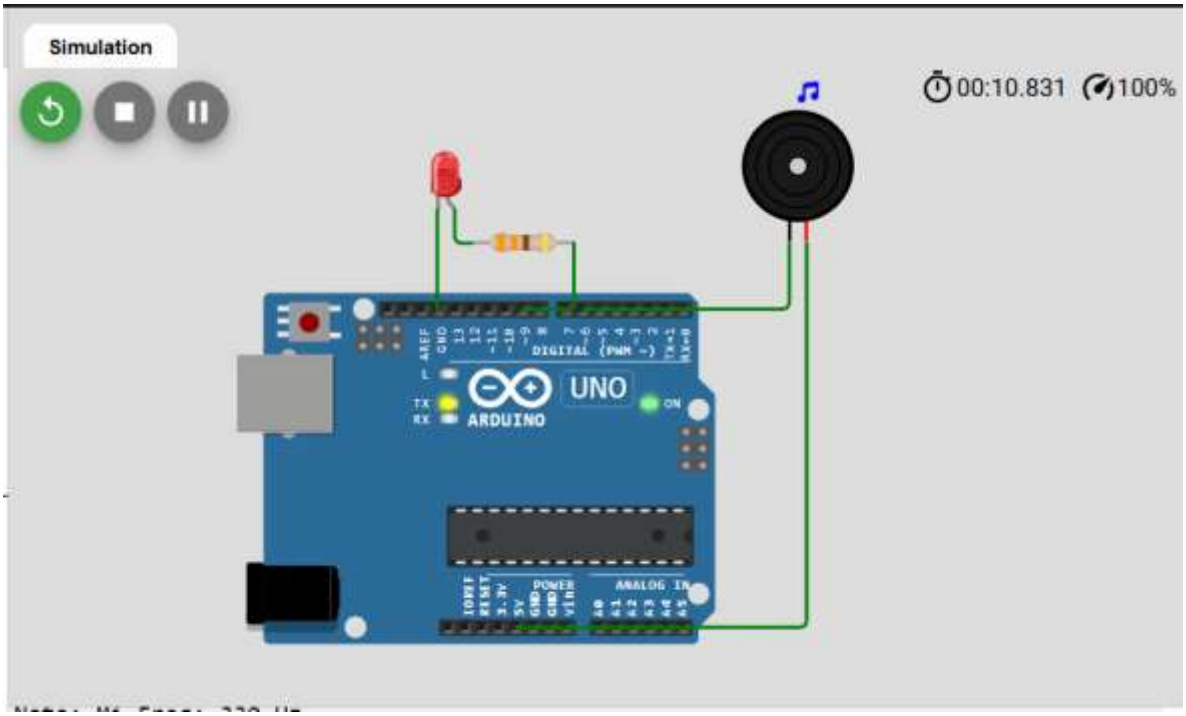
PARTE 2: Efecto 'Breathing' con función seno




```
// Lab03_Breathing_LED.ino

#include <math.h>
const int PIN_LED = 9;
float angulo = 0.0;

void setup() {
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  // sin() devuelve -1.0 a 1.0 — mapear a 0-255
  float seno = (sin(angulo) + 1.0) / 2.0; // 0.0 a 1.0
  int brillo = (int)(seno * 255);
```

```
  analogWrite(PIN_LED, brillo);

  angulo += 0.05; // Velocidad del efecto
  if (angulo > 2 * PI) angulo = 0;
  delay(15);
}
```

PARTE 3: Buzzer: tonos con tone()

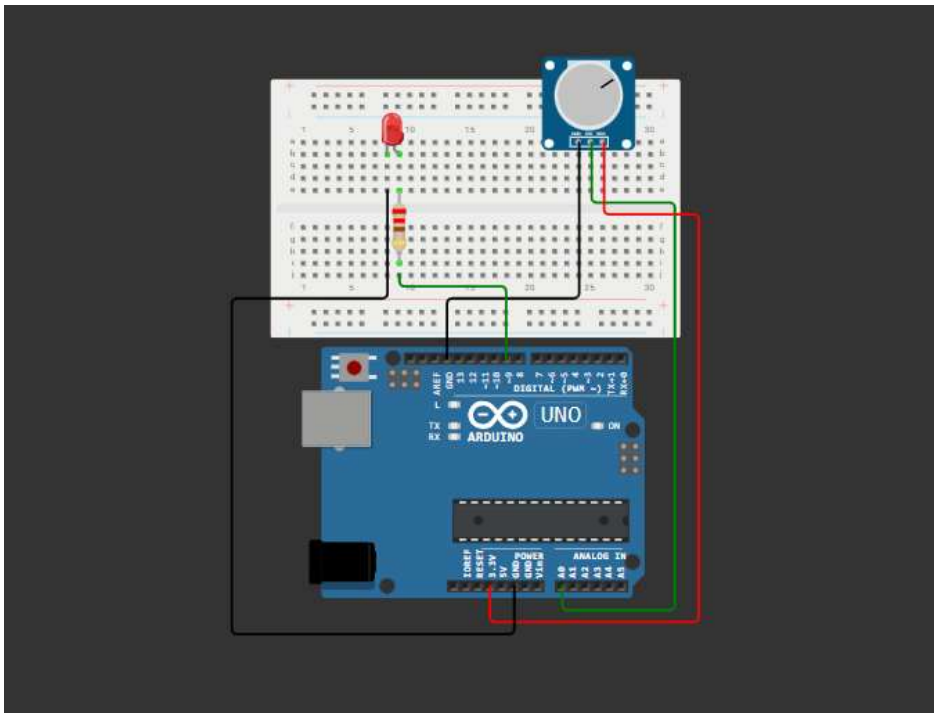


```
// Lab03_Buzzer_Melodia.ino

const int PIN_BUZZER = 6;

// Frecuencias de notas (Do-Re-Mi-Fa-Sol-La-Si)
int notas[] = {262, 294, 330, 349, 392, 440, 494};
```

```
String nombres[] = {"Do","Re","Mi","Fa","Sol","La","Si"};  // ✓ comillas dobles
int duracion = 300;

void setup() {
  Serial.begin(9600);
  Serial.println("Lab 03 - Buzzer: escala musical");  // ✓ comillas dobles
}

void loop() {
  for (int i = 0; i < 7; i++) {
    Serial.print("Nota: ");
    Serial.print(nombres[i]);
    Serial.print(" Freq: ");
    Serial.print(notas[i]);
    Serial.println(" Hz");
    tone(PIN_BUZZER, notas[i], duracion);
    delay(duracion + 50);
  }

  noTone(PIN_BUZZER);
  delay(1000);
}
```

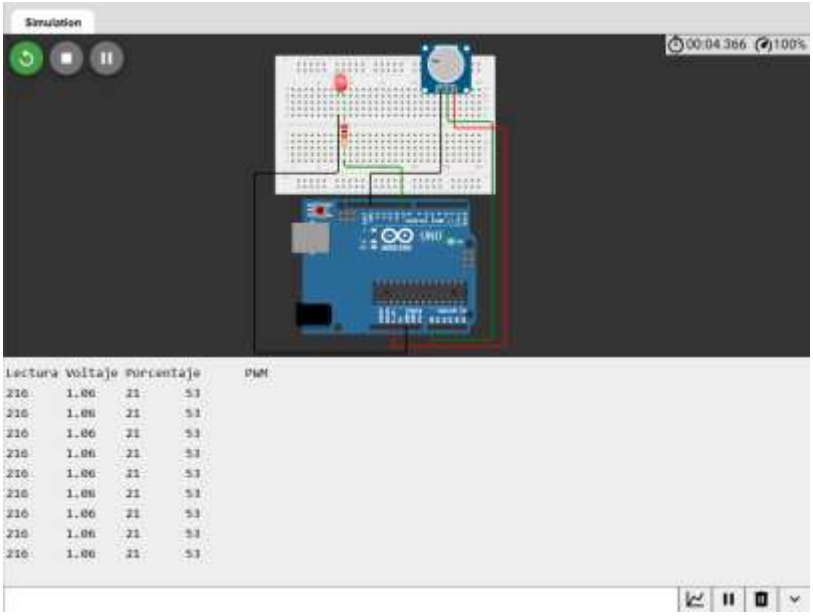
PRACTICA 4.- Sensores Analógicos — ADC, LDR y Potenciómetro

PARTE 1: Potenciómetro: divisor de voltaje variable

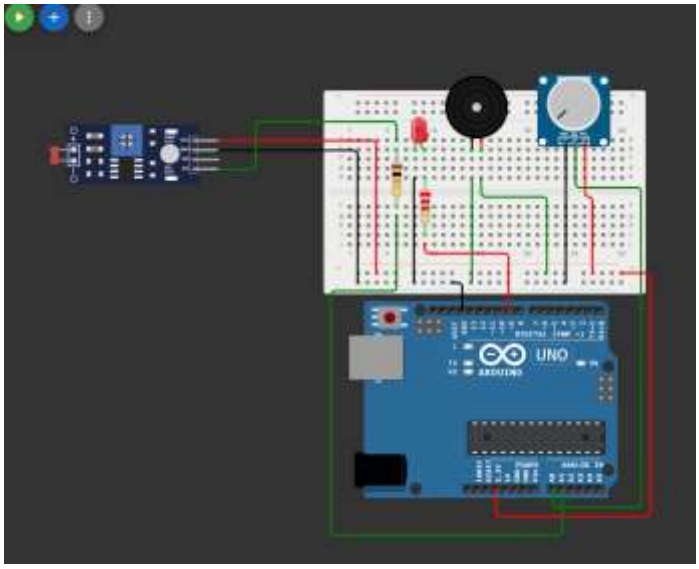



```
const int PIN_POT = A0;
const int PIN_LED = 9;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  Serial.println("Lectura\tVoltaje\tPorcentaje\tPWM");

}

void loop() {
```

```
  // put your main code here, to run repeatedly:
  int lectura = analogRead(PIN_POT);
  float voltaje = lectura * (5.0 / 1023.0);
  int porcentaje = map(lectura, 0, 1023, 0, 100);
  int pwm_val = map(lectura, 0, 1023, 0, 255);

  analogWrite(PIN_LED, pwm_val);

  Serial.print(lectura); Serial.print("\t");
  Serial.print(voltaje, 2); Serial.print("\t");
  Serial.print(porcentaje); Serial.print("\t");
  Serial.println(pwm_val);

  delay(500);

}
```

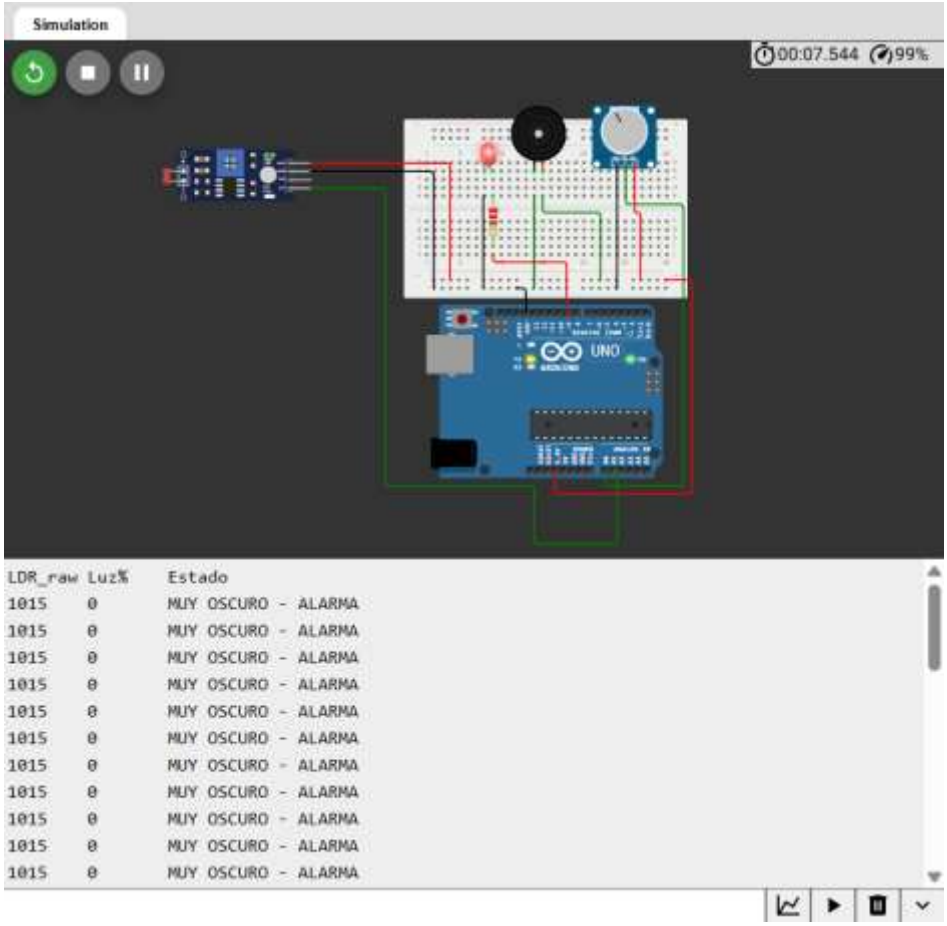


PARTE 2: LDR: sensor de luz



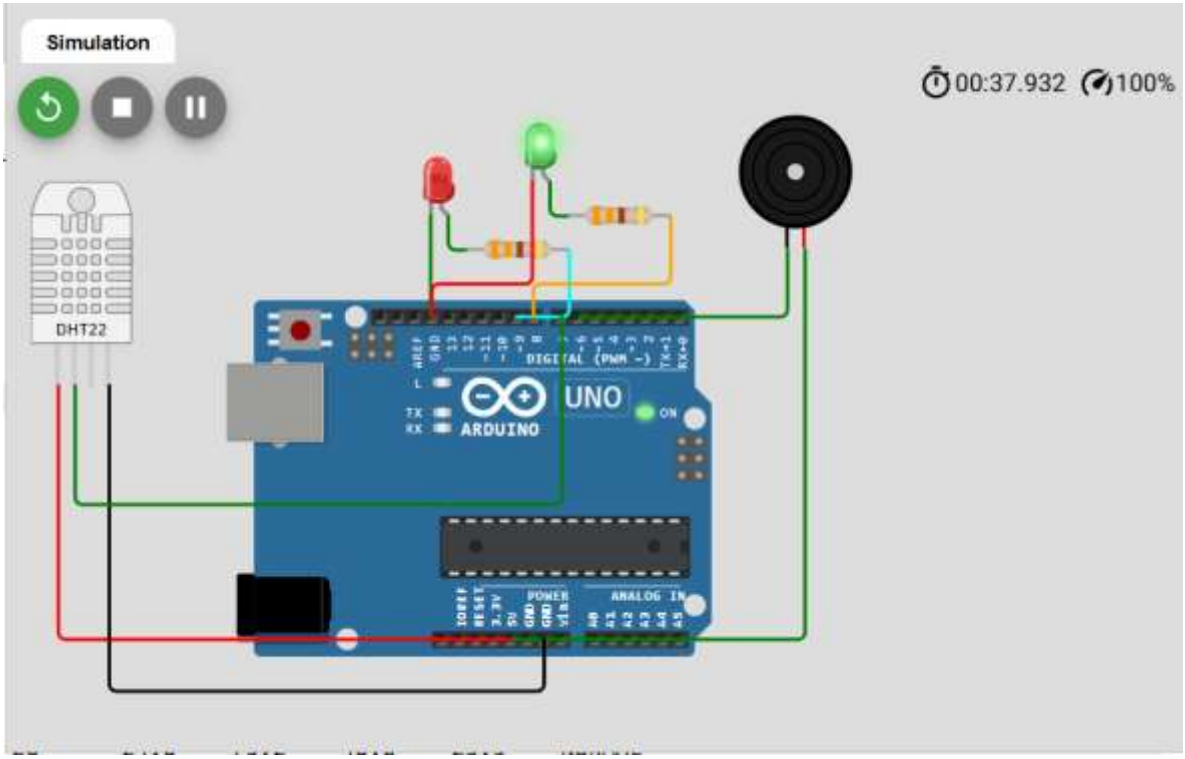

```
const int PIN_LDR = A1;
const int PIN_LED = 9;
const int PIN_BUZZER  = 6;

const int LDR_OSCURO = 800;
const int LDR_LUZ = 100;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  Serial.println("LDR_raw\tLuz%\tEstado");

}

void loop() {
  // put your main code here, to run repeatedly:
```

```
  int lectura_ldr = analogRead(PIN_LDR);

  int luz_pct = map(lectura_ldr, LDR_OSCURO, LDR_LUZ, 0, 100);
  luz_pct = constrain(luz_pct, 0, 100);

  int brillo_led = map(luz_pct, 0, 100, 255, 0);
  analogWrite(PIN_LED, brillo_led);

  String estado;
  if(luz_pct < 20){
    estado = "MUY OSCURO - ALARMA";
    tone(PIN_BUZZER, 1000, 100);
  } else if(luz_pct < 50){
    estado = "PENUMBRA";
  } else{
    estado = "ILUMINADO";
  }

  Serial.print(lectura_ldr); Serial.print("\t");
  Serial.print(luz_pct); Serial.print("\t");
  Serial.println(estado);
  delay(200);
}
```


Simulation
00:07.544
99%
UNO
LDR_raw Luz% Estado
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA
1015 0 MUY OSCURO - ALARMA

PRACTICA 5.- Sensor de Temperatura y Humedad DHT22

PARTE 1: Instalación de librería y conexión



22	24.0	75.2	40.0	23.5	NORMAL
24	24.0	75.2	40.0	23.5	NORMAL
26	24.0	75.2	40.0	23.5	NORMAL
28	24.0	75.2	40.0	23.5	NORMAL
30	24.0	75.2	40.0	23.5	NORMAL
32	24.0	75.2	40.0	23.5	NORMAL
34	24.0	75.2	40.0	23.5	NORMAL
36	24.0	75.2	40.0	23.5	NORMAL
38	24.0	75.2	40.0	23.5	NORMAL
40	24.0	75.2	40.0	23.5	NORMAL
42	24.0	75.2	40.0	23.5	NORMAL
44	24.0	75.2	40.0	23.5	NORMAL
46	24.0	75.2	40.0	23.5	NORMAL
48	24.0	75.2	40.0	23.5	NORMAL
50	24.0	75.2	40.0	23.5	NORMAL

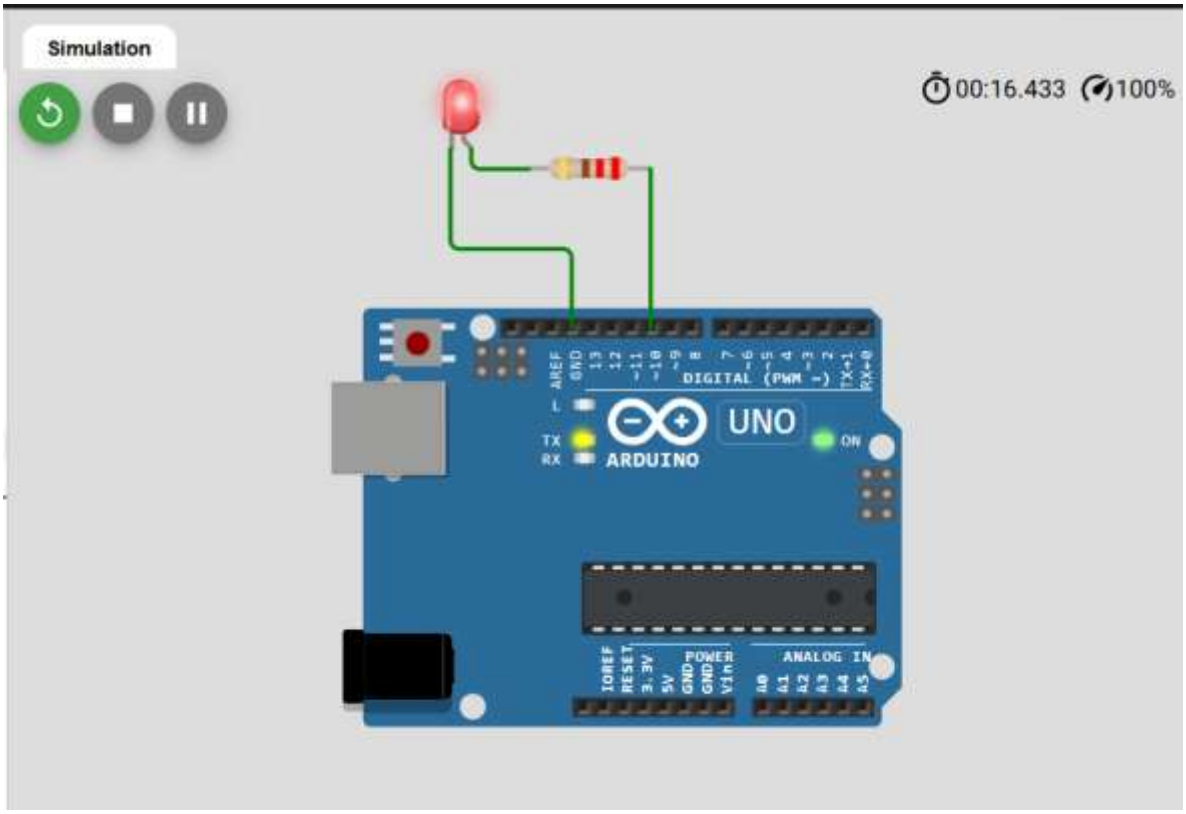
```
// Lab05_DHT22_Basico.ino

#include "DHT.h"

#define PIN_DHT        7
#define TIPO_DHT       DHT22
#define PIN_LED_VERDE  8
#define PIN_LED_ROJO   9
#define PIN_BUZZER     6

const float TEMP_ALTA = 30.0;
const float TEMP_BAJA = 10.0;
const float HUM_ALTA  = 80.0;

DHT dht(PIN_DHT, TIPO_DHT);

void setup() {
  Serial.begin(9600);
  dht.begin();

  pinMode(PIN_LED_VERDE, OUTPUT);
  pinMode(PIN_LED_ROJO,  OUTPUT);

  Serial.println("Tiempo(s)\tTemp(C)\tTemp(F)\tHumedad%\tHeatIdx\tEstado");
}
```

```
void loop() {
  delay(2000);

  float humedad = dht.readHumidity();
  float temp_c  = dht.readTemperature();
  float temp_f  = dht.readTemperature(true);

  if (isnan(humedad) || isnan(temp_c)) {
    Serial.println("ERROR: Fallo en lectura DHT22");
    digitalWrite(PIN_LED_ROJO, HIGH);
    return;
  }

  float heat_idx = dht.computeHeatIndex(temp_c, humedad, false);

  String estado;

  if (temp_c > TEMP_ALTA || humedad > HUM_ALTA) {
    estado = "ALERTA-CALOR";
    digitalWrite(PIN_LED_ROJO,  HIGH);
    digitalWrite(PIN_LED_VERDE, LOW);
    tone(PIN_BUZZER, 2000, 200);

  } else if (temp_c < TEMP_BAJA) {
    estado = "ALERTA-FRIO";
```

```
    digitalWrite(PIN_LED_ROJO,  HIGH);
    digitalWrite(PIN_LED_VERDE, LOW);
    tone(PIN_BUZZER, 500, 500);

  } else {
    estado = "NORMAL";
    digitalWrite(PIN_LED_VERDE, HIGH);
    digitalWrite(PIN_LED_ROJO,  LOW);
  }

  Serial.print(millis() / 1000);  Serial.print("\t");
  Serial.print(temp_c,   1);      Serial.print("\t");
  Serial.print(temp_f,   1);      Serial.print("\t");
  Serial.print(humedad,  1);      Serial.print("\t");
  Serial.print(heat_idx, 1);      Serial.print("\t");
  Serial.println(estado);
}
```

PRACTICA 6.- Actuadores — Motor DC y Servomotor

PARTE 1: Por qué usar un driver de motor




```
Velocidad: 210
Velocidad: 220
Velocidad: 230
Velocidad: 240
Velocidad: 250
Atras: velocidad media...
Adelante: velocidad creciente...
Velocidad: 0
Velocidad: 10
Velocidad: 20
Velocidad: 30
```

// Lab06_Motor_DC.ino

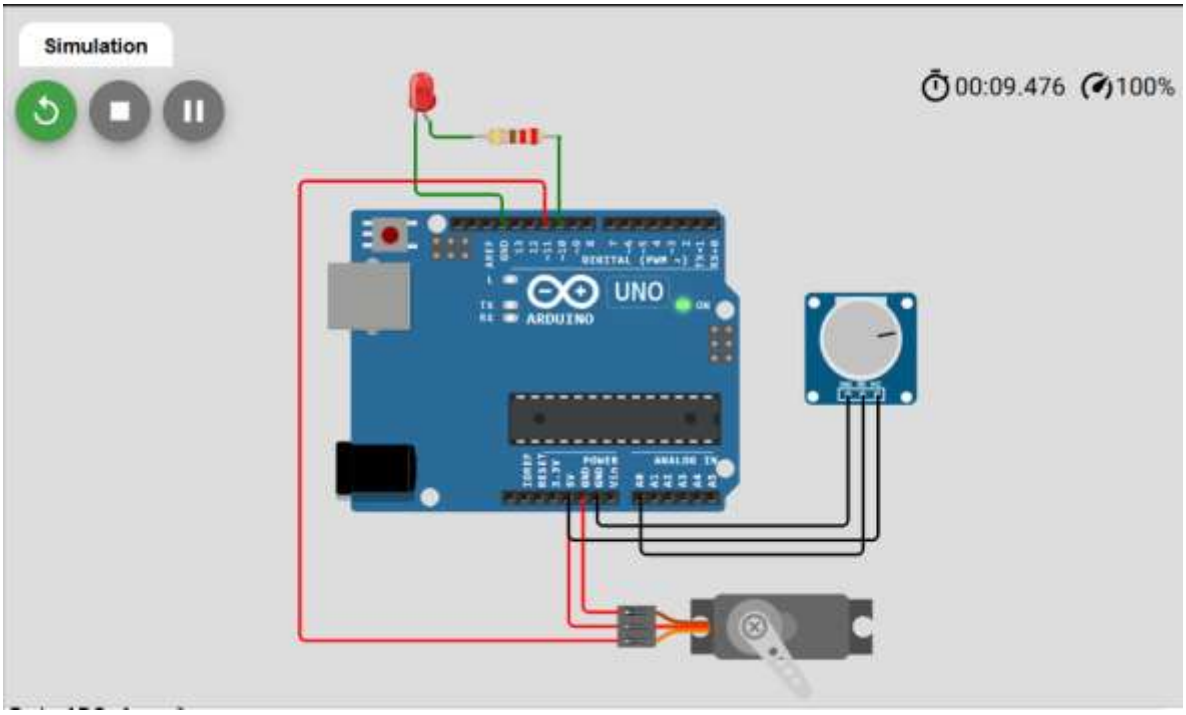
```
const int ENA = 10;
const int IN1 = 8;
const int IN2 = 9;
```

```
void motorAdelante(int velocidad) {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, velocidad);
}

void motorAtras(int velocidad) {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, velocidad);
}

void motorFreno() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, 255);
}

void motorLibre() {
  analogWrite(ENA, 0);
}

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
```

```
  pinMode(IN2, OUTPUT);

  Serial.begin(9600);
  Serial.println("Lab 06 - Control Motor DC");
}

void loop() {

  Serial.println("Adelante: velocidad creciente...");

  for (int v = 0; v <= 255; v += 10) {
    motorAdelante(v);
    Serial.print("Velocidad: ");
    Serial.println(v);
    delay(300);
  }

  motorLibre();
  delay(500);

  Serial.println("Atras: velocidad media...");
  motorAtras(180);
  delay(2000);

  motorFreno();
  delay(500);
```

```
  delay(1000);
}
```

PARTE 2: Servomotor con librería Servo.h




```
Servo listo. Gira el potenciometro.
Pot_ADC Angulo
318     55
955     168
928     163
841     147
777     136
765     134
788     138
0       0
1023    180
0       0
30      5
45      7
```

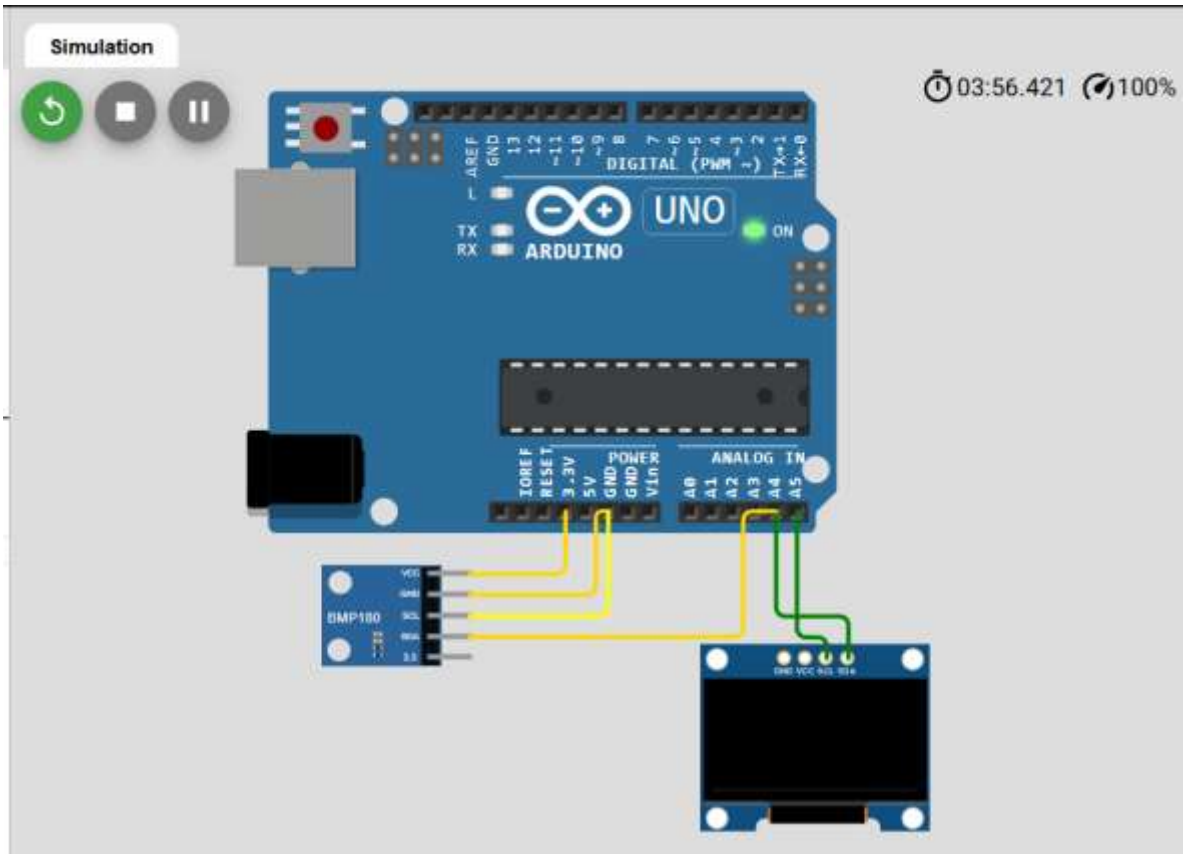
```
#include <Servo.h>

Servo mi_servo;

const int PIN_SERVO = 11;
```

```
const int PIN_POT = A0;

int angulo_anterior = -1;

void setup() {
  mi_servo.attach(PIN_SERVO);
  mi_servo.write(90);

  Serial.begin(9600);
  Serial.println("Servo listo. Gira el potenciometro.");
  Serial.println("Pot_ADC\tAngulo");
}

void loop() {
  int pot_val = analogRead(PIN_POT);

  int angulo = map(pot_val, 0, 1023, 0, 180);

  if (abs(angulo - angulo_anterior) >= 2) {
    mi_servo.write(angulo);
    angulo_anterior = angulo;

    Serial.print(pot_val);
    Serial.print("\t");
    Serial.println(angulo);
  }
```

```
  delay(20);
}
```

PRACTICA 7.- Comunicación I2C — Sensor BMP280 y Pantalla OLED

PARTE 1: Escaneo del bus I2C



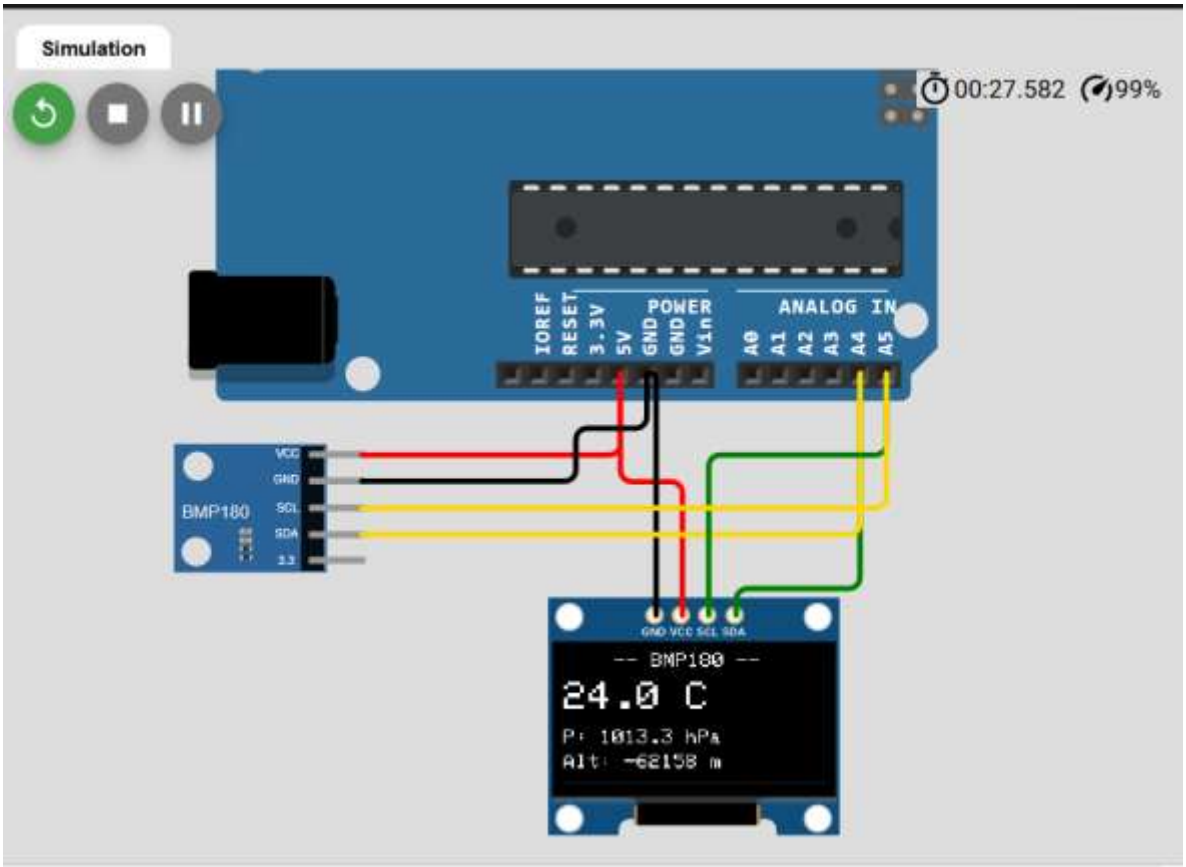

```
Escaneando bus I2C...
Rango: 0x08 - 0x77

  Dispositivo en: 0x3C

Total de dispositivos: 1
```

```
// Lab07_I2C_Scanner.ino
#include <Wire.h>

void setup() {
```

```
Wire.begin();
Serial.begin(9600);
Serial.println("Escaneando bus I2C...");
Serial.println("Rango: 0x08 - 0x77");
Serial.println();

int dispositivos = 0;

for (byte addr = 8; addr < 120; addr++) {
  Wire.beginTransmission(addr);
  byte error = Wire.endTransmission();

  if (error == 0) {
    dispositivos++;
    Serial.print("  Dispositivo en: 0x");
    if (addr < 16) Serial.print("0");
    Serial.println(addr, HEX);
  }
}

Serial.println();
Serial.print("Total de dispositivos: ");
Serial.println(dispositivos);

if (dispositivos == 0)
  Serial.println("Ninguno encontrado. Verifica conexiones y pull-ups.");
```

```
}

void loop() {}
```

PARTE 2: Lectura BMP280 + Display OLED




```
BMP180 y OLED listos.
Tiempo  Temp(C) Presion(hPa)    Altitud(m)
0       24.00   1013.27 -62157.7
1       24.00   1013.27 -62157.7
2       24.00   1013.27 -62157.7
3       24.00   1013.27 -62157.7
4       24.00   1013.27 -62157.7
5       24.00   1013.27 -62157.7
6       24.00   1013.27 -62157.7
```

// Lab07_BMP180_OLED.ino

```
#include <Wire.h>

#include <Adafruit_BMP085.h>

#include <Adafruit_SSD1306.h>
```

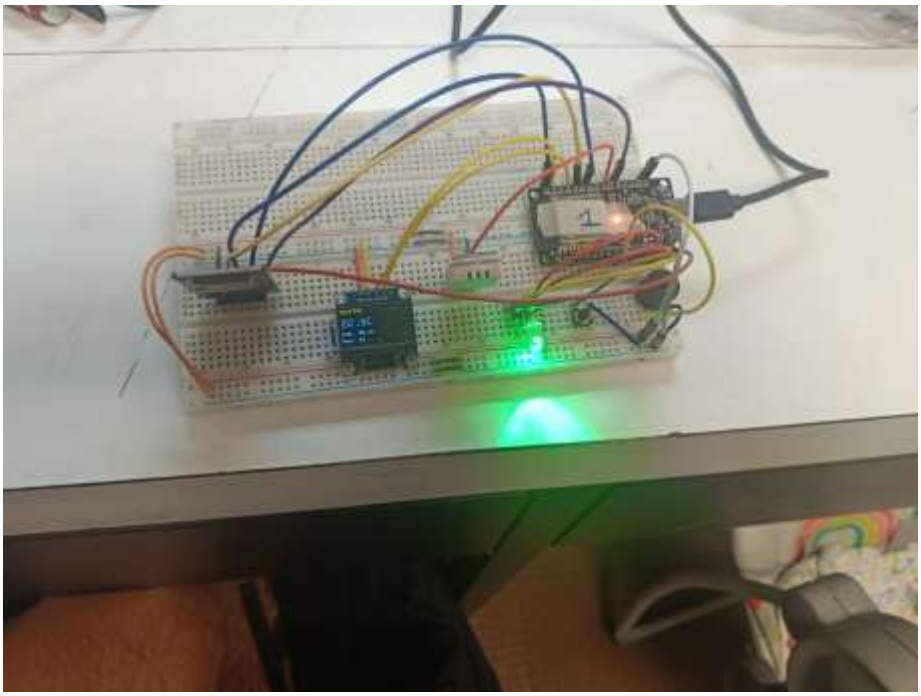
```
#include <Adafruit_GFX.h>

#define OLED_W    128
#define OLED_H     64
#define OLED_ADDR  0x3C

Adafruit_BMP085 bmp;
Adafruit_SSD1306 display(OLED_W, OLED_H, &Wire, -1);

const float PRESION_MAR = 1013.25;

// ──────────────────────────────────────────────────────────────
void setup() {
  Serial.begin(9600);

  if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("ERROR: OLED no encontrada");
    while (true);
  }

  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(10, 28);
  display.println("Iniciando BMP180...");
  display.display();
```

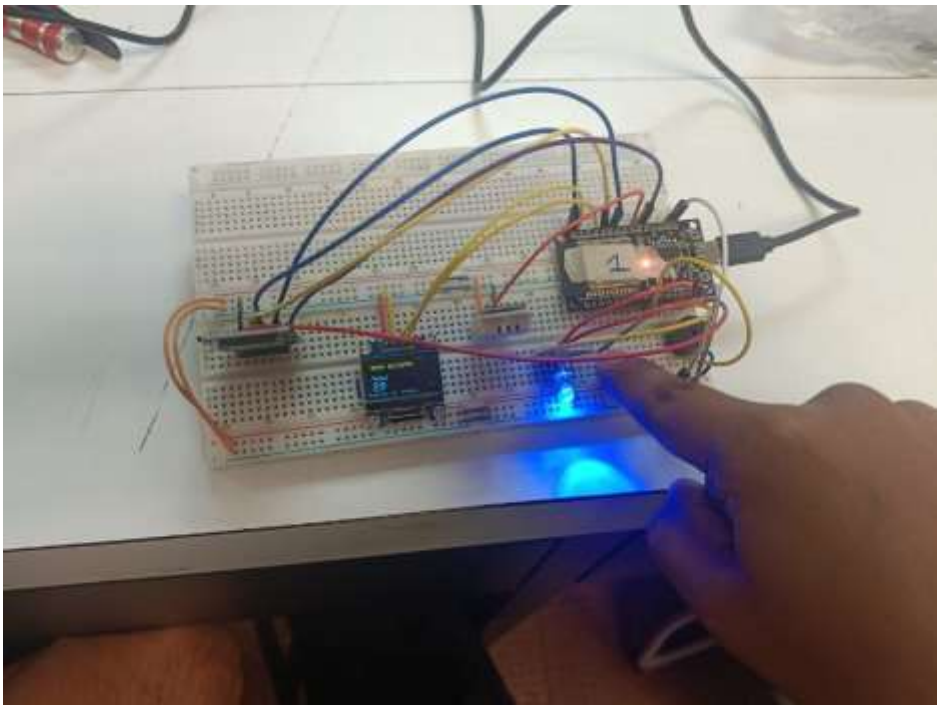
```
  if (!bmp.begin()) {
    Serial.println("ERROR: BMP180 no encontrado");
    display.clearDisplay();
    display.setCursor(10, 28);
    display.println("ERROR: BMP180");
    display.display();
    while (true);
  }

  Serial.println("BMP180 y OLED listos.");
  Serial.println("Tiempo\tTemp(C)\tPresion(hPa)\tAltitud(m)");
}

void loop() {
  float temp    = bmp.readTemperature();
  float presion = bmp.readPressure() / 100.0;
  float altitud = bmp.readAltitude(PRESION_MAR);

  display.clearDisplay();

  display.setTextSize(1);
  display.setCursor(25, 0);
  display.println("-- BMP180 --");

  // Temperatura
```

```
display.setTextSize(2);
display.setCursor(0, 14);
display.print(temp, 1);
display.println(" C");

// Presión
display.setTextSize(1);
display.setCursor(0, 38);
display.print("P: ");
display.print(presion, 1);
display.println(" hPa");

// Altitud
display.setCursor(0, 50);
display.print("Alt: ");
display.print(altitud, 0);
display.println(" m");

display.display();

// ── Monitor Serial ──────────────────────────────
Serial.print(millis() / 1000); Serial.print("\t");
Serial.print(temp, 2);         Serial.print("\t");
Serial.print(presion, 2);      Serial.print("\t");
Serial.println(altitud, 1);
```

```
  delay(1000);
}
```

PRACTICA 8.- Comunicación SPI y Proyecto Integrador Bloque I





```
#include <Wire.h>
#include <SPI.h>
#include <SD.h>
#include <DHT.h>
#include <Adafruit_SSD1306.h>

#define PIN_DHT     16
#define PIN_CS_SD   4
#define PIN_BTN     26
#define PIN_LED_R   13
#define PIN_LED_G   12
#define PIN_LED_B   14
#define PIN_BUZZER  27

DHT dht(PIN_DHT, DHT22);
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

enum Estado { MONITOR, ALERTA, LOG };
Estado estado_actual = MONITOR;

float temp_c = 0, humedad = 0;
unsigned long t_ultimo_guardado = 0;
unsigned long t_btn_presionado  = 0;
int total_registros = 0;
bool oled_ok = false;
const int INTERVALO_GUARDADO = 10000;
```

```
void setLED(bool r, bool g, bool bl) {
  digitalWrite(PIN_LED_R, r  ? HIGH : LOW);
  digitalWrite(PIN_LED_G, g  ? HIGH : LOW);
  digitalWrite(PIN_LED_B, bl ? HIGH : LOW);
}

void guardarEnSD() {
  File f = SD.open("/datos.csv", FILE_APPEND);
  if (f) {
    f.print(millis() / 1000); f.print(",");
    f.print(temp_c, 1);       f.print(",");
    f.println(humedad, 1);
    f.close();
    total_registros++;
    Serial.print("Guardado #"); Serial.println(total_registros);
  }
}

void mostrarOLED() {
  if (!oled_ok) return;
  oled.clearDisplay();
  oled.setTextColor(WHITE);
  oled.setTextSize(1);

  switch (estado_actual) {
```

```
  case MONITOR:
    oled.setCursor(0, 0);  oled.print("MONITOR");
    oled.setTextSize(2);
    oled.setCursor(0, 16); oled.print(temp_c, 1); oled.print("C");
    oled.setTextSize(1);
    oled.setCursor(0, 40); oled.print("Hum: "); oled.print(humedad, 1);
oled.print("%");
    oled.setCursor(0, 54); oled.print("Reg: "); oled.print(total_registros);
    break;
  case ALERTA:
    oled.setCursor(20, 0);  oled.print("!!! ALERTA !!!");
    oled.setTextSize(2);
    oled.setCursor(0, 16); oled.print(temp_c, 1); oled.print("C");
    oled.setTextSize(1);
    oled.setCursor(0, 42);
    if (temp_c > 35)  oled.print("TEMPERATURA ALTA");
    if (humedad > 85) oled.print("HUMEDAD ALTA");
    break;
  case LOG:
    oled.setCursor(10, 10); oled.print("MODO REGISTRO");
    oled.setCursor(10, 28); oled.print("Total: ");
    oled.setTextSize(2);
    oled.setCursor(10, 40); oled.print(total_registros);
    oled.setTextSize(1);
    oled.setCursor(0, 58);  oled.print("Suelta el boton");
    break;
 }
```

```
  oled.display();
}

void setup() {
  Serial.begin(115200);

  pinMode(PIN_LED_R, OUTPUT);
  pinMode(PIN_LED_G, OUTPUT);
  pinMode(PIN_LED_B, OUTPUT);
  setLED(false, false, false);

  pinMode(PIN_CS_SD, OUTPUT);
  digitalWrite(PIN_CS_SD, HIGH);

  Wire.begin(21, 22);
  if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("Error: OLED no encontrada.");
  } else {
    oled_ok = true;
    Serial.println("OLED lista.");
    oled.clearDisplay();
    oled.setTextColor(WHITE);
    oled.setTextSize(1);
    oled.setCursor(0, 0); oled.print("Iniciando...");
    oled.display();
  }
```

```
  dht.begin();
  pinMode(PIN_BTN, INPUT_PULLUP);

  if (SD.begin(PIN_CS_SD)) {
    Serial.println("SD lista.");
  } else {
    Serial.println("Error: SD no encontrada.");
  }

  Serial.println("Sistema listo.");
}

void loop() {
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (!isnan(h) && !isnan(t)) { humedad = h; temp_c = t; }

  bool btn = (digitalRead(PIN_BTN) == LOW);
  if (btn) {
    if (t_btn_presionado == 0) t_btn_presionado = millis();
    if (millis() - t_btn_presionado > 3000) estado_actual = LOG;
  } else {
    t_btn_presionado = 0;
    if (estado_actual == LOG) estado_actual = MONITOR;
  }
```

```
if (estado_actual != LOG) {
  if (temp_c > 35.0 || humedad > 85.0)
    estado_actual = ALERTA;
  else if (estado_actual == ALERTA)
    estado_actual = MONITOR;
}

switch (estado_actual) {
  case MONITOR:
    setLED(false, true, false);
    Serial.print("MONITOR | Temp: "); Serial.print(temp_c);
    Serial.print("C | Hum: "); Serial.print(humedad);
    Serial.print("% | Reg: "); Serial.println(total_registros);
    if (millis() - t_ultimo_guardado >= INTERVALO_GUARDADO) {
      guardarEnSD();
      t_ultimo_guardado = millis();
    }
    break;
  case ALERTA:
    setLED(true, false, false);
    tone(PIN_BUZZER, 1500, 300);
    Serial.print("!!! ALERTA !!! Temp: "); Serial.print(temp_c);
    Serial.print("C | Hum: "); Serial.println(humedad);
    break;
  case LOG:
```

```
    setLED(false, false, true);
    Serial.print("MODO LOG | Total: "); Serial.println(total_registros);
    break;
  }

  mostrarOLED();
  delay(500);
}
```